

# Сторожев Иван Андреевич

20 лет

Java разработчик

Опыт: 7 месяца

Местонахождение - Россия, Москва

Английский: B1

**Образование** - Московский технический университет связи и информатики

Факультет: Информатика и вычислительная техника

Направление: Машинное обучение

## Навыки и компетенции

### Общие навыки:

- Проектирование и разработка микросервисной и монолитной архитектуры
- Оптимизация производительности сервисов
- Интеграция с внешними API
- Разработка и поддержка RESTful и gRPC API
- Разработка тестов (модульные, интеграционные)
- Разработка CI/CD пайплайнов
- Контейнеризация и оркестрация
- Настройка мониторинга сервисов
- Код-ревью, работа с git-flow
- Вдумчивый подход к задачам, глубокое погружение в контекст
- Следование принципам чистой архитектуры и чистого кода
- Четкое аргументирование технических решений
- Менторство младших разработчиков

### Языки программирования:

- Java

### Фреймворки:

- Spring Boot (Core, Data, Security, WebFlux, Cloud)
- Hibernate, JPA
- JUnit, Mockito, Testcontainers

### Базы данных:

- PostgreSQL, MySQL, Redis, MongoDB, S3 хранилище

## **Технологии и инструменты:**

- Kafka, RabbitMQ
- Liquibase, Flyway
- Prometheus, Grafana, ELK
- JWT, Keycloak, OAuth2

## **Средства разработки ПО:**

- IntelliJ IDEA, Git, Docker, Kubernetes, Jira, Confluence
- Gradle, Maven
- GitLab CI/CD

## **ПОСЛЕДНИЕ ПРОЕКТЫ**

### ***Effective-mobile***

**Проект:** Админка и бот для автоматизации регистрации новых сотрудников и обучающихся.

**Период:** Август 2025 - Декабрь 2025

**Роль:** Java Developer

**Описание проекта:** Проект разрабатывался для отделов HR и менеджмента с целью полной автоматизации онбординга новых сотрудников и предоставление доступов ученикам. В рамках проекта была реализована централизованная админ-панель, предоставляющая единый интерфейс для управления жизненным циклом сотрудника и ученика: от первичной регистрации и назначения ответственного руководителя до автоматизированного создания корпоративной почты и предоставления доступов к внутренним ресурсам компании для обучения.

Для удобства сотрудников и учеников был разработан чат-бот, выступающий в роли персонального ассистента. Через бота сотрудники и ученики могут получать актуальные ссылки на корпоративные сервисы, запрашивать свои данные и информацию о корпоративной почте, что значительно снизило нагрузку на техническую поддержку.

Ключевой особенностью сервиса является интеграция с LDAP-системой, которая выступает единым источником истины для управления доступом и правами пользователей, обеспечивая безопасность и согласованность данных во всех связанных системах.

## **Состав команды:**

- 3 Backend-разработчик
- 1 DevOps инженер
- 2 Аналитика
- 1 PO

#### **Стек:**

- **Язык программирования:** Java 17
- **Архитектура:** Модульный монолит
- **Фреймворки:** Spring Boot, Spring Data, Vaadin
- **Базы данных:** PostgreSQL, Redis
- **Контейнеризация:** Docker
- **CI/CD:** GitLab CI/CD
- **Мониторинг:** Prometheus, Grafana
- **Интеграции:** LDAP, Telegram Bot API
- **Сборка:** Maven

#### **Обязанности:**

- Разработка серверной логики и API
- Участие в проектировании и реализация модульного монолита
- Интеграция с корпоративными системами
- Внедрение практик Observability
- Обеспечение надежности развертывания
- Автоматизация процессов разработки (CI/CD)

#### **Достижения:**

- Обнаружил критический баг, связанный с отсутствием идемпотентности в админ панели. Обеспечил идемпотентность с помощью идемпотентного ключа.
- Настройка сбора метрик приложения (JVM, HTTP-запросы, кастомные метрики) с помощью Prometheus, создание дашбордов в Grafana для мониторинга системы и оперативного реагирования на инциденты. Организация централизованного сбора и анализа логов через logstash.
- Покрыл модуль с ботом unit тестами, что позволило довести coverage до 80.
- Реализовал логику истории действий пользователей в админке через AOP и outbox pattern, что позволило делать аудит действий в реальном времени.
- Реализовал маппинг доступных ресурсов в боте, что позволило динамически генерировать нужные ссылки на ресурсы пользователям

#### **Ажур**

**Проект:** Сайт для кейтеринга

**Период:** Июнь 2025 - Август 2025

**Роль:** Fullstack Developer

**Описание проекта:** Разработка полнофункционального веб-сайта для компании, предоставляющей кейтеринговые услуги.

**Состав команды:**

- 1 Fullstack-разработчик
- 1 Frontend-разработчик

**Стек:**

- **Язык программирования:** Java 21
- **Архитектура:** Модульный монолит
- **Фреймворки:** Spring Boot, Spring Data
- **Базы данных:** PostgreSQL, Redis, S3-хранилище
- **Контейнеризация:** Docker
- **CI/CD:** GitHub Actions
- **Мониторинг:** Prometheus, Grafana, ELK
- **Сборка:** Maven

**Обязанности:**

- Разработка и проектирование серверной архитектуры
- Создание и интеграция REST API
- Реализация подсистемы безопасности
- Организация работы с бинарными данными
- Интеграция email-уведомлений через SMTP
- Внедрение систем мониторинга и observability
- Контейнеризация и развертывание
- Автоматизация процессов разработки
- Координация с фронтенд

**Достижения:**

- Спроектировал и реализовал масштабируемую backend-архитектуру в парадигме модульного монолита на Java 21 с использованием Spring Boot, что позволило четко разделить модули и упростить поддержку кода.
- Разработал полный цикл REST API для взаимодействия с клиентской частью, обеспечив эффективную обработку данных и бизнес-логики.
- Внедрил и настроил механизм JWT-аутентификации для защиты маршрутов API и обеспечения безопасного доступа пользователей к ресурсам.

- Реализовал надежный и производительный модуль для загрузки, хранения и управления файлами (изображениями) с использованием облачного S3-хранилища.
- Настроил систему отправки транзакционных писем для регистрации (код подтверждения) и обработки заявок.
- Настроил сбор метрик приложения и инфраструктуры с помощью Prometheus, визуализацию в Grafana и централизованное логирование через ELK-стек для оперативного контроля за состоянием системы.
- Обеспечил переносимость и стабильность окружения за счет полной контейнеризации приложения и его зависимостей с использованием Docker.
- Организовал процесс непрерывной интеграции и доставки (CI/CD) через GitHub Actions, что ускорило выкладку обновлений и повысило их надежность.
- Обеспечил тесную интеграцию между backend-сервисами и фронтенд-частью приложения, участвуя в согласовании API-контрактов и форматов данных.

## ПЕТ-ПРОЕКТЫ

### *StreamFlow (видео-сервис)*

**Проект:** Сайт для просмотра и выкладки видео.

**Описание проекта:** Видеоплатформа на микросервисной архитектуре, аналог YouTube. Проект включает backend на Java (Spring Boot), frontend на React с TypeScript и инфраструктуру на Docker.

Backend состоит из микросервисов. Video Service - основной: загружает видео (до 10GB), обрабатывает через FFmpeg (транскодирование в 240p–1080p, генерация превью), хранит в MinIO, управляет метаданными в PostgreSQL, индексирует в Elasticsearch для поиска, обрабатывает асинхронно через Kafka. Gateway Service - единая точка входа, маршрутизация и балансировка. Discovery Service (Eureka) - регистрация и обнаружение сервисов. Comment Service - комментарии. Subscription Service - подписки. Notification Service - уведомления. Recommendation Service (Python/FastAPI) - рекомендации.

Инфраструктура: PostgreSQL (основная БД), Elasticsearch + Logstash + Kibana (поиск и логи), Redis (кеш), MinIO (объектное хранилище), Kafka (асинхронная обработка), Keycloak (OAuth2/OIDC), все в Docker Compose.

Основные процессы: загрузка видео (валидация, сохранение в MinIO, создание записи в БД), асинхронная обработка (Kafka → транскодирование через FFmpeg → генерация превью → сохранение в MinIO → обновление статуса), поиск (индексация в Elasticsearch, полнотекстовый поиск с русской морфологией), просмотр (выбор качества, временные URL из MinIO, аналитика просмотров).

Безопасность: OAuth2/OIDC через Keycloak, JWT, защита эндпоинтов Spring Security, валидация файлов, проверка прав доступа.

Технические решения: асинхронная обработка через Kafka, кеширование в Redis, Circuit Breaker (Resilience4j), разделение ответственности между сервисами, версионирование БД через Liquibase, оптимизация запросов (пагинация, индексы), временные URL для раздачи контента.

## **О СЕБЕ**

Java-разработчик с опытом участия в коммерческих проектах по созданию высоконагруженных бэкенд-систем. Владею глубоким пониманием принципов чистой архитектуры, проектирования масштабируемых API (REST, gRPC) и оптимизации производительности. Имею практический опыт внедрения полноценных CI/CD-процессов, систем мониторинга (Observability) и контейнеризации приложений. Отличаюсь вдумчивым подходом к задачам, умением быстро погружаться в сложную предметную область и находить эффективные решения. Стремлюсь к разработке качественного, поддерживаемого кода и активному профессиональному росту в команде сильных специалистов.

### **Самая технически сложная задача:**

Обеспечение идемпотентности и согласованности данных при массовых операциях в HR-админке с интеграцией в LDAP.

**Задача:** В админ-панели существовала функция массового назначения доступов к ресурсам группам сотрудников. При сетевых сбоях или таймаутах LDAP-операции HR-менеджеры повторяли запрос, что приводило к дублированию назначений, несогласованному состоянию систем и необходимости ручного разбора данных техподдержкой. Требовалось гарантировать, что каждая операция, даже повторная, выполняется ровно один раз и сохраняет согласованность между внутренней БД и внешней LDAP-системой.

### **Решение:**

Ввел концепцию идемпотентного ключа (idempotency-key), генерируемого на клиенте для каждой серии массовых операций. Реализовал на уровне сервиса проверку ключа в Redis (как быстрое распределенное хранилище) с TTL 24 часа. Повторный запрос с тем же ключом возвращал результат первоначальной операции без выполнения бизнес-логики.

Реализовал строгий порядок действий. Сначала все данные проверялись и фиксировались в нашей базе PostgreSQL. Только после успешного сохранения

выполнялись соответствующие операции в LDAP. Для сбоев при работе с LDAP добавил автоматические повторные попытки

**Результат:**

- Устранение дублирования: Полностью исключены случаи повторного назначения прав, что ликвидировало 95% связанных с этим инцидентов.
- Снижение нагрузки на техподдержку: Количество обращений от HR по вопросам дублирования прав и несогласованности данных сократилось до нуля.
- Улучшение UX: Пользователи админки получили четкую обратную связь о статусе массовых операций и возможность безопасно повторять запросы без риска побочных эффектов.